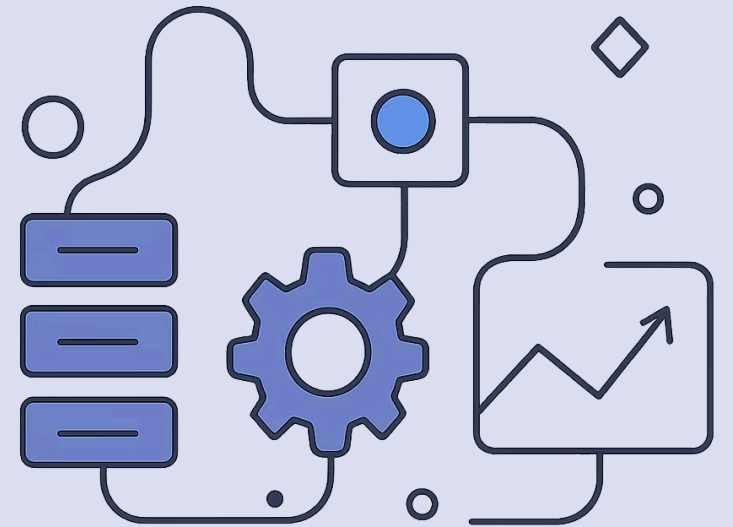


Scaling, Normalization, and Encoding

Preparing Data for Later Workflows

A beginner's guide to transforming clean data into model-ready columns



Why Preprocessing Matters

Preprocessing converts columns into useful forms for later workflows.

Two common needs arise:

Numeric Scale

monthly_spend: 700 to 1600

visits_per_month: 1 to 6

Text Categories

membership: Gold, Silver, Bronze — need numeric representation

- ❏ Preprocessing prepares columns for the next step.



Cleaned vs. Preprocessed Data

Cleaned Data

- Human-readable
- Corrected for basic quality problems
- Useful for EDA and reports

Preprocessed Data

- Transformed for later workflows
- May include scaled numeric columns
- May include encoded category columns
- Useful for modeling and repeatable pipelines

 Cleaned data is understandable. Preprocessed data is usable by algorithms.

Column Preparation Map

Different columns need different preparation.

Column Type	Example	Common Preparation
Identifier	customer_id	Keep for reference, not a model feature
Text label	name	Keep for readability, usually not a feature
Numeric	monthly_spend	Scale or normalize
Category	city	One-hot encode
Ordered category	membership	Ordered mapping if order is meaningful

Why We Need Scaling: A Numeric Example

Machine learning algorithms, especially those relying on distance calculations (like K-Nearest Neighbors or Support Vector Machines), are sensitive to the magnitude of feature values.

Consider two features: `annual_salary` and `years_experience`.

Customer A:

`annual_salary` = \$70,000

`years_experience` = 5

Customer B:

`annual_salary` = \$72,000

`years_experience` = 8

The difference in `annual_salary` is \$2,000. The difference in `years_experience` is 3 years.

Without scaling, an algorithm would perceive the \$2,000 difference as significantly more impactful than the 3 years difference, even if both are equally important for the task.

 Unscaled features with larger numeric ranges can dominate calculations, making other important features seem insignificant.

Original Values Still Matter

Original Values

Easy to explain to humans:

monthly_spend = 1200 rupees

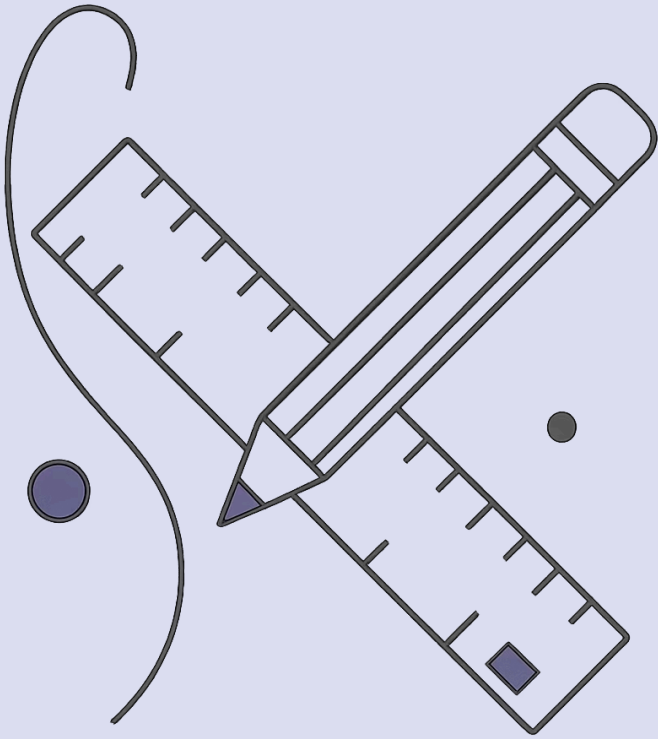
visits_per_month = 4 visits

Scaled Values

Useful for algorithms:

monthly_spend_standard_scaled = 0.35

- ❗ Keep readable columns while learning so you can interpret results. Preprocessing helps workflows, but original units help humans understand.



Standard Scaling

Distance From Average

Standard scaling changes values so the mean is near 0 and the standard deviation is near 1.

□ Formula: standard scaled value = $(\text{value} - \text{mean}) / \text{standard deviation}$

Simple meaning: *How far is this value from average?*

Reading Standard Scaled Values

Value	Simple Meaning
0	Close to average
Positive	Above average
Negative	Below average
Large absolute value	Farther from average

-1.2

Below average

0.0

Near average

1.5

Above average

 Standard scaled values can be negative. That is expected.

Standard Scaling Example

Monthly Spend

Monthly Spend values: 700, 850, 1200, 1450, 1600

Step 1 – Calculate the Mean

Mean = (700 + 850 + 1200 + 1450 + 1600) / 5 = 5800 / 5 = 1160

Step 2 – Calculate the Standard Deviation

Std Dev ≈ 340

Step 3 – Apply the formula

$z = (\text{value} - \text{mean}) / \text{std}$

Original Value	Calculation	Scaled Value
700	$(700 - 1160) / 340$	-1.35
850	$(850 - 1160) / 340$	-0.91
1200	$(1200 - 1160) / 340$	+0.12
1450	$(1450 - 1160) / 340$	+0.85
1600	$(1600 - 1160) / 340$	+1.29

How to read the result

- 700 → -1.35 means this customer spends well below average
- 1200 → +0.12 means this customer spends just slightly above average
- 1600 → +1.29 means this customer spends well above average

📌 Notice: the scaled values are centered around 0. Negative = below average, Positive = above average.

Python code

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['monthly_spend_scaled']] = scaler.fit_transform(df[['monthly_spend']])
```

Min-Max Normalization

Position Between Minimum and Maximum

Min-max normalization changes values to a fixed range, usually 0 to 1.

☐ Formula: $\text{normalized value} = (\text{value} - \text{minimum}) / (\text{maximum} - \text{minimum})$

Simple meaning: *Where does this value sit between the smallest and largest value?*

Monthly Spend Example

Monthly Spend values: 700, 850, 1200, 1450, 1600

Step 1 – Find the Minimum and Maximum

Min = 700

Max = 1600

Step 2 – Apply the formula

$\text{normalized} = (\text{value} - \text{min}) / (\text{max} - \text{min})$

Original Value	Calculation	Normalized Value
700	$(700 - 700) / (1600 - 700)$	0.00
850	$(850 - 700) / (1600 - 700)$	0.17
1200	$(1200 - 700) / (1600 - 700)$	0.56
1450	$(1450 - 700) / (1600 - 700)$	0.83
1600	$(1600 - 700) / (1600 - 700)$	1.00

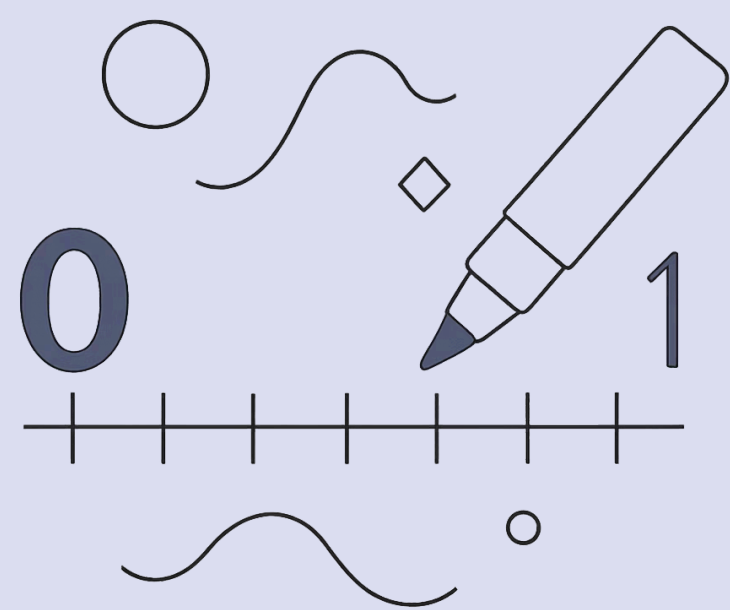
How to read the result

- 700 → 0.00 means this is the lowest spender (at the bottom of the range)
- 1200 → 0.56 means this customer is about halfway between min and max
- 1600 → 1.00 means this is the highest spender (at the top of the range)

☐ All values are now between 0 and 1. 0 = minimum, 1 = maximum, 0.5 = exactly halfway.

Python code

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['monthly_spend_normalized']] =
scaler.fit_transform(df[['monthly_spend']])
```



Reading Normalized Values

Value	Simple Meaning
0	Smallest value in that column
1	Largest value in that column
0.5	Around the middle of min and max
Between 0 and 1	Relative position in the range

 0.75 means closer to the maximum than the minimum. Normalized values should stay between 0 and 1 when using the same fitted range.

Example: Interpreting the normalized monthly_spend values

Recall from the previous slide:

- Original values: 700, 850, 1200, 1450, 1600
- Normalized values: 0.00, 0.17, 0.56, 0.83, 1.00

How to read each normalized value:

Customer	Original Spend	Normalized Value	Interpretation
A	700	0.00	At the minimum — lowest spender in the dataset
B	850	0.17	17% of the way from min to max — still a low spender
C	1200	0.56	56% of the way from min to max — slightly above middle
D	1450	0.83	83% of the way from min to max — high spender
E	1600	1.00	At the maximum — highest spender in the dataset

 A value of 0.56 means this customer is positioned 56% of the way between the lowest and highest spender.

Key insight: The normalized value tells you the customer's position along the spending range, like a percentage from bottom (0%) to top (100%).

Scaling vs. Normalization

Similar Goal, Different Meaning

Method	Result	Simple Meaning
Standard scaling	Mean ≈ 0 , std dev ≈ 1	How far from average?
Min-max normalization	Values between 0 and 1	Where between min and max?
Original values	Actual business units	What does this mean in real life?

❏ Scaling compares distance from average. Normalization compares position in a range.

When to Use Which?

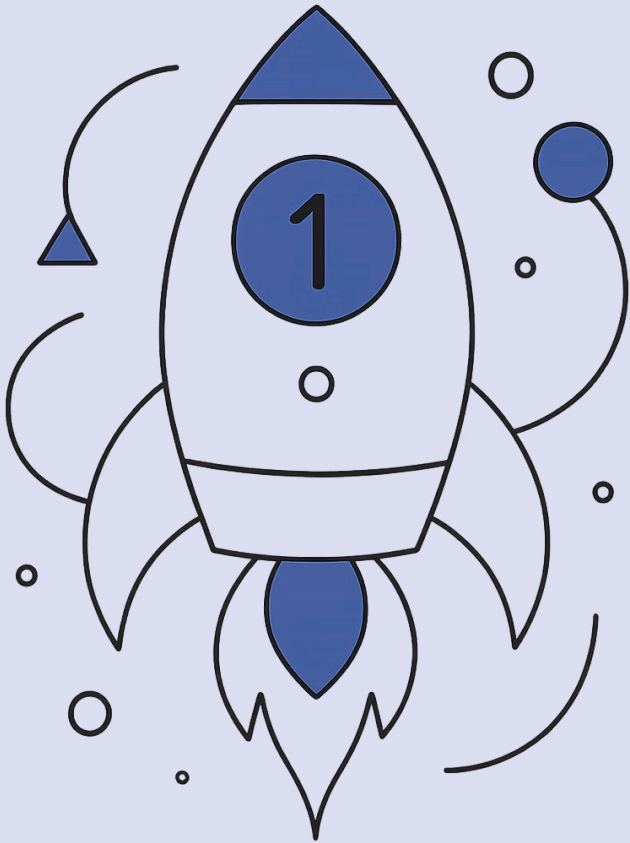
Use Standard Scaling When

- You care about distance from average
- Later methods are sensitive to scale
- Values may not need a strict 0–1 range

Use Min-Max Normalization When

- You want values in a fixed 0–1 range
- You want simple relative position
- The minimum and maximum are meaningful

 For Day 1, the goal is to understand both ideas — not memorize every modeling rule.



Encoding

Turning Categories Into Numbers

Many later workflows need numbers.

Text categories like Gold, Silver, Bronze or Mumbai, Delhi, Chennai need a numeric representation.

❏ Encoding makes categories usable for numeric workflows.

Why Not Just Number Every Category?

The Fake Order Problem

Bad Idea

Mumbai = 1

Delhi = 2

Chennai = 3

This implies Chennai > Delhi > Mumbai.

The Problem

City names have no natural order.

Assigning numbers creates false distance that can mislead later calculations.

⊗ Do not create fake order where no real order exists.

One-Hot Encoding

Separate 0/1 Columns

One-hot encoding creates one column per category. Each row gets 1 for the matching city and 0 for all others.

City	city_Mumbai	city_Delhi	city_Chennai
Mumbai	1	0	0
Delhi	0	1	0
Chennai	0	0	1

📌 Each city becomes its own yes/no column. One-hot encoding avoids fake order.

Validate One-Hot Encoding

Check the Rows

For one-hot encoding of one column, each row should usually have exactly one 1.

$\text{city_Mumbai} + \text{city_Delhi} + \text{city_Chennai} = 1$

If the sum is not 1, investigate:

- Category may be missing
- Encoding may be incomplete
- There may be multi-label data

❏ After encoding, check that the rows still make sense.

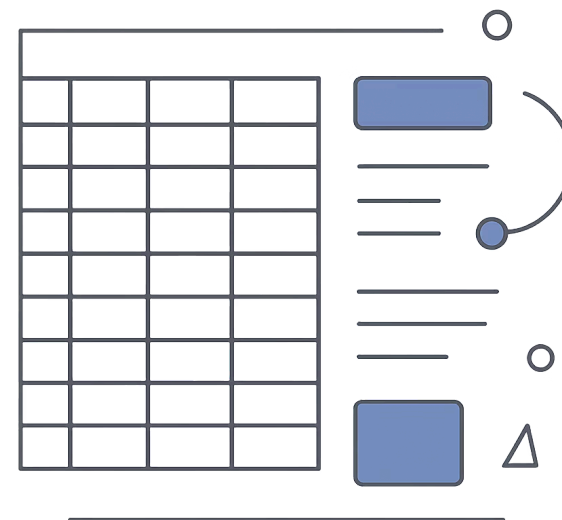
Encoding Multiple Columns

Columns to Encode

- City
- Membership
- Signup month

Result: original categorical columns become many 0/1 columns. The dataset expands horizontally.

 Many categories can create many columns — be aware of dataset width.



Encoding Multiple Columns – Example

Here is a concrete example with three customers and three categorical columns.

Original Data table

Customer	City	Membership	Signup Month
A	Mumbai	Gold	Jan
B	Delhi	Bronze	Mar
C	Chennai	Silver	Jan

After one-hot encoding, each category value gets its own binary column.

After Encoding table

Custo mer	city_M umbai	city_D elhi	city_C henna i	memb ership _Gold	memb ership _Bron ze	memb ership _Silver	signu p_Jan	signu p_Mar
A	1	0	0	1	0	0	1	0
B	0	1	0	0	1	0	0	1
C	0	0	1	0	0	1	1	0

✔ Original: 3 categorical columns. After encoding: 8 binary columns. The dataset expanded horizontally from 3 categorical columns to 8 binary columns.

```
df_encoded = pd.get_dummies(df, columns=['city', 'membership', 'signup_month'])
print(df_encoded.shape) # More columns than before
```

ℹ Each original categorical column becomes multiple 0/1 columns — one per unique category value.

Ordered Mapping

Use Only When Order Is Real

Good Example

Membership has meaningful rank:

Bronze = 1

Silver = 2

Gold = 3

Bronze < Silver < Gold — the order is real.

When to Use It

Ordered mapping is useful only when the category order is real and meaningful for later calculations.



Do not force numeric order onto categories that have no natural rank.

Ordered Mapping Caution

Do Not Force Order

Bad Mapping

Mumbai = 1
Delhi = 2
Chennai = 3

Why It's Wrong

- Cities have no numeric rank
- Numbers imply false distance
- Later calculations may be misleading

⊗ Only ordered categories should get ordered numbers.

Unknown Categories

What If a Category Is New?

The Scenario

Training data cities: Mumbai, Delhi, Chennai

New data city: Pune

Questions to consider:

- Should it become "Unknown"?
- Should the encoder handle unknown values?
- Should the category list be updated?



⚠ Preprocessing decisions must handle future data, not only today's file.

Clean Before Encoding

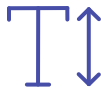
Missing Categories Before Encoding

Before encoding, check for category quality issues:



Missing Values

Null or blank category cells



Casing & Spaces

Gold, gold, GOLD — standardize first



Spelling Variants

Unexpected labels or typos

⊗ Encoding preserves category problems if you do not clean first.

Row Alignment

Keep Rows Matched Correctly

When combining transformed columns:

- Row order must remain the same
- Index alignment matters
- Row count should match original data
- No accidental missing values should appear

☐ Good check: original rows == preprocessed rows



A preprocessing table is only useful if the transformed values still belong to the correct rows.

Row Alignment - Example

Here’s an example showing how rows must stay aligned when original data is transformed. The Customer ID links the original and preprocessed data.

Original Data

Customer ID	Product Category	Price
101	Electronics	1200
102	Books	30
103	Clothing	80

Preprocessed Data

After scaling the 'Price' and one-hot encoding 'Product Category':

Customer ID	Price_scaled	Product_Books	Product_Clothing	Product_Electronic s
101	0.95	0	0	1
102	0.01	1	0	0
103	0.05	0	1	0

✔ Each row of preprocessed data still corresponds to the correct original customer, ensuring data integrity for downstream analysis.

Preprocessed Table

What Should It Include?

Readable Columns

- Customer ID
- Name
- Original city and membership

Transformed Columns

- Standard-scaled numeric columns
- Normalized numeric columns
- One-hot encoded category columns
- Ordered membership level

☐ Keep readable columns while learning, and add transformed columns beside them.

Quality Checks Before Saving

Verify the Prepared Data

1

Row & Column Count

Rows match original? Column count expected?

2

Missing Values

No nulls introduced by transformations?

3

Scaled Value Summary

Normalized values in 0-1 range? Scaled values centered near 0?

4

Encoded Column Names

Ordered mapping missing values checked?



Preprocessing is incomplete until the prepared table is checked.



Save Preprocessed Data

Create a Handoff File

Save prepared data separately as
`customer_activity_preprocessed.csv`

- Reusable
Later notebooks can load it directly
- Repeatable
The workflow becomes a consistent pipeline
- Separated
Raw, cleaned, and preprocessed files stay distinct

Common Beginner Mistakes

Scaling Confusion

Thinking scaling and normalization are the same; forgetting scaled values can be negative

Encoding Errors

Creating fake order for unordered categories; encoding before cleaning text

Table Integrity

Forgetting row alignment; dropping readable columns too early; skipping checks before saving

⊗ Preprocessing changes data — every change needs a reason and a check.



Customer Activity Preparation

Simple Example Summary

Original Columns

- monthly_spend
- visits_per_month
- city
- membership

Prepared Columns

- monthly_spend_standard_scaled
- visits_per_month_normalized
- city_Mumbai, city_Delhi, city_Chennai
- membership_level

❏ The prepared table contains the same customers, but the columns are easier for later workflows to use.

Discussion Prompt

Choose the Transformation

Should city be encoded with one-hot encoding or ordered mapping?

Expected Answer

One-hot encoding — city has no natural order.

Follow-Up

Should membership be one-hot or ordered mapped?

Either can work, but ordered mapping is reasonable if Bronze < Silver < Gold is meaningful.



Section Summary

What We Learned

Preprocessing

Prepares columns for later workflows — not just human readability

Scaling

Standard scaling = distance from average.
Normalization = position in range. Fit learns; transform applies.

Encoding

One-hot avoids fake order. Ordered mapping only for real order. Clean before encoding.

Verify and Save

Check rows, missing values, and column names. Save preprocessed data separately.

✔ Good preprocessing makes later analysis safer and more repeatable.

Key Memory Lines

“

Preprocessing prepares columns for the next step.

”

“

Standard scaling answers: how far from average?
Normalization answers: where between min and max?

”

“

Fit learns. Transform applies.

”

“

One-hot encoding avoids fake order. Only ordered categories should get ordered numbers.

”

“

Separate files preserve the project journey.

”

Quick Reference

Scaling, Normalization, and Encoding at a Glance

Concept	Applies To	Key Idea
Standard scaling	Numeric	Distance from average; mean ≈ 0
Min-max normalization	Numeric	Position in range; values 0–1
fit / transform	Scalers	Learn settings, then apply them
One-hot encoding	Unordered categories	Separate 0/1 columns; no fake order
Ordered mapping	Ordered categories	Only when rank is real (Bronze < Silver < Gold)
Save preprocessed file	Final step	Separate file for later workflows

Clean data is readable. Preprocessed data is ready.

You have completed Section 6. Your data is now prepared for later analysis and machine learning workflows.

SECTION 6 COMPLETE

